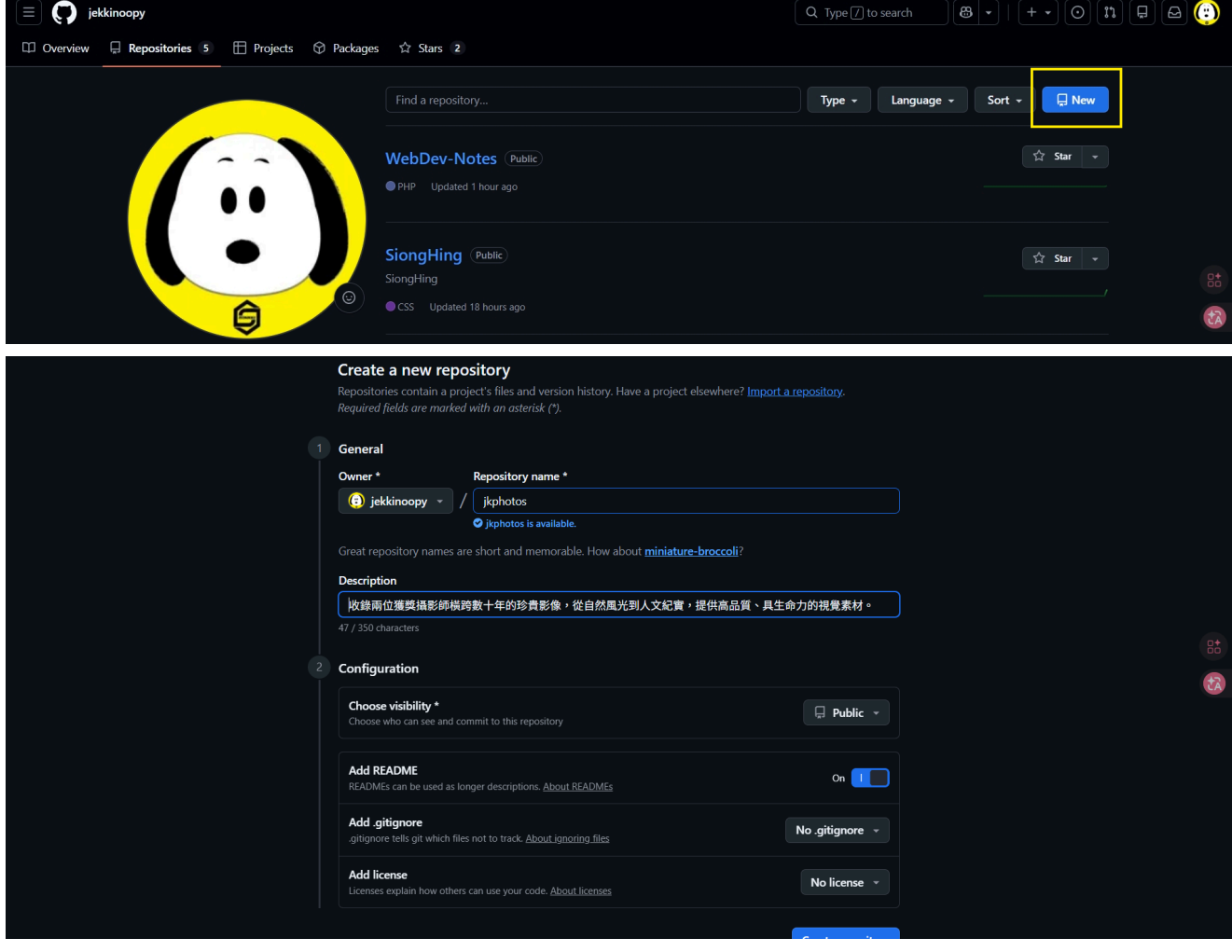


專案初始化 (Project Initialization)

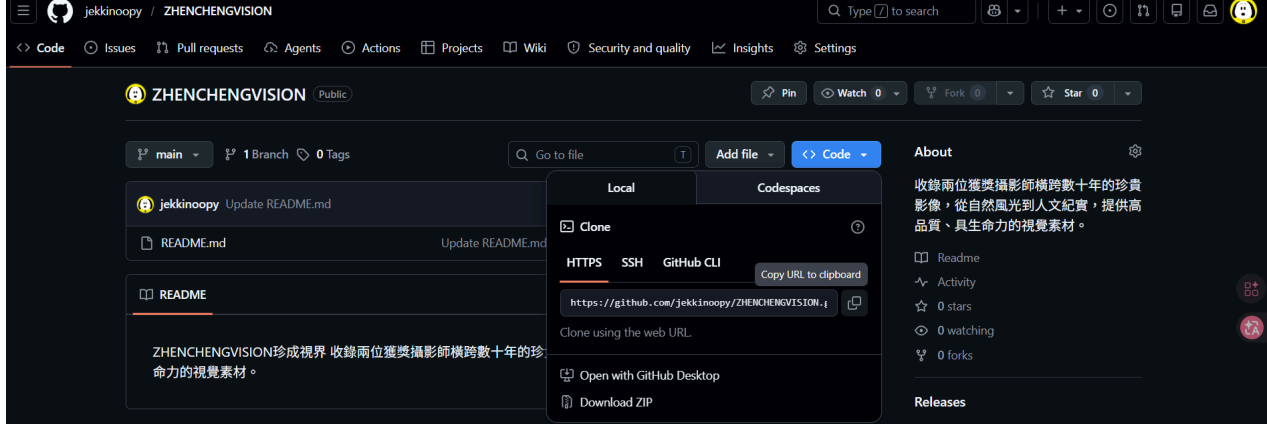
你在網頁上進行的這個完整流程，術語叫做「**初始化一個遠端倉庫**」(Initializing a Remote Repository)。這是在雲端 (GitHub) 建立一個空間，用來存放並管理你未來所有的程式碼版本。



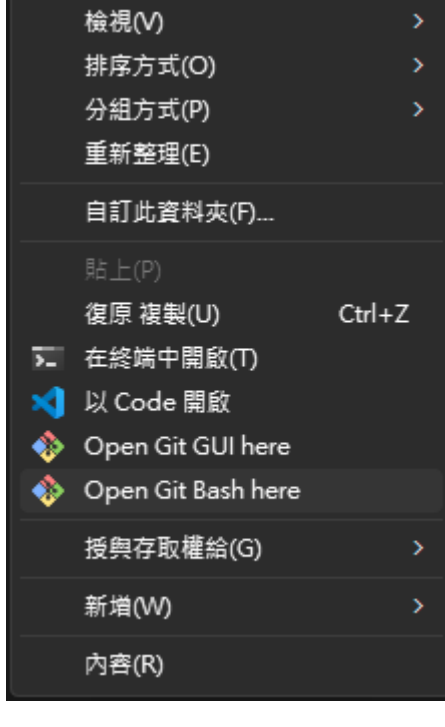
Clone (複製)

如果你電腦裡還沒有這個專案的資料夾，你要使用這個網址將雲端的內容整份下載到本地。

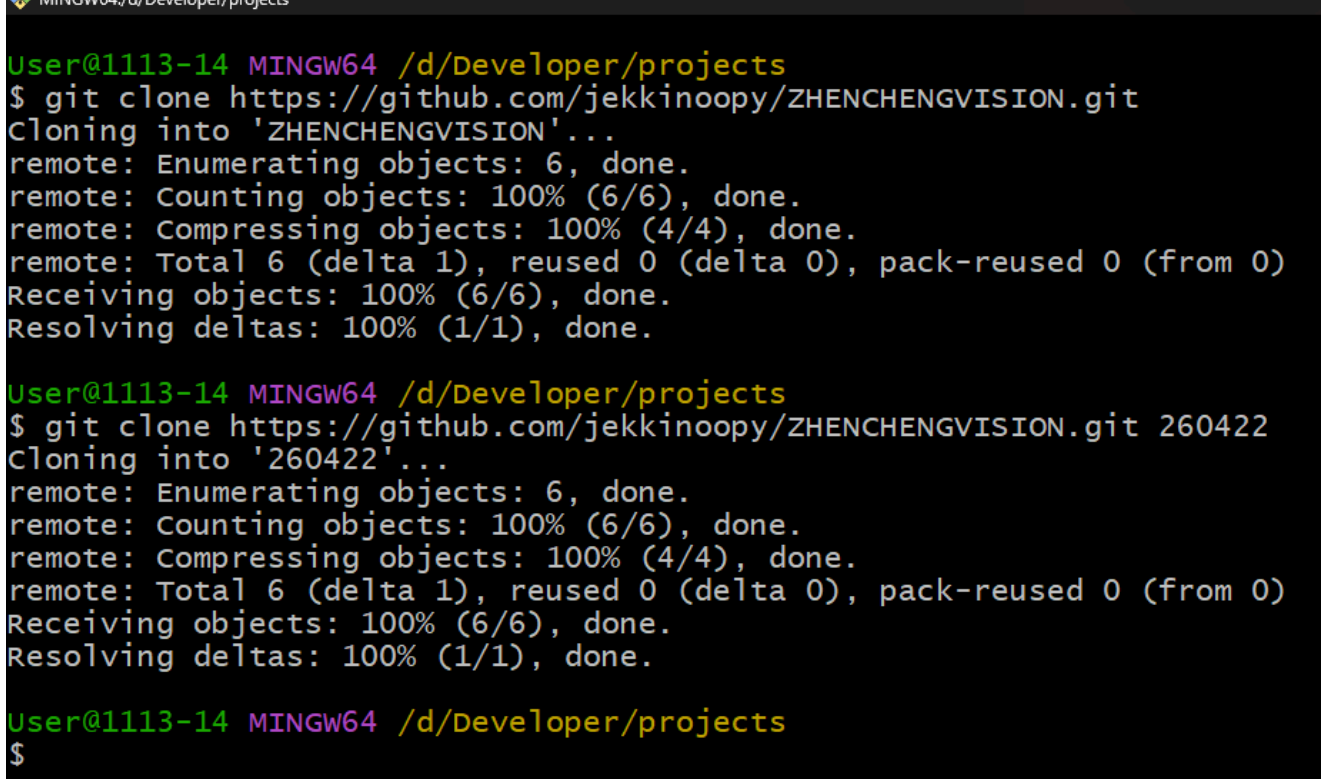
- 指令：`git clone <網址>`
- 定義：將遠端倉庫 (Remote Repository) 完整複製一份到本地端 (Local) 的動作。



<https://github.com/jekkinooopy/ZHENCHENGVISION.git>



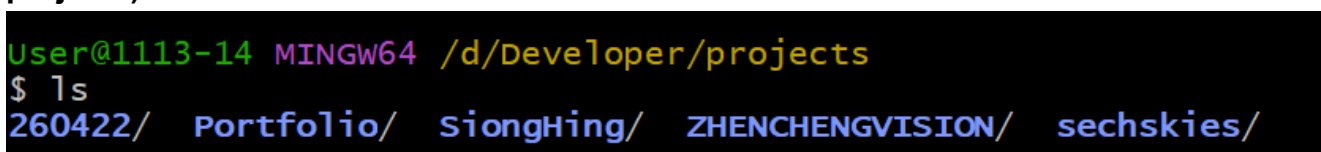
- `git clone https://github.com/jekkinooopy/ZHENCHENGVISION.git`
- `git clone https://github.com/jekkinooopy/ZHENCHENGVISION.git 260422`



列出目錄內容

執行 `ls` (List segments，意指列出目錄內容) 後，我們可以看到你目前手頭上的所有開發專案。

這種列出所有專案資料夾的動作，在開發者日常中稱為「**檢視專案清單**」(Listing projects)。

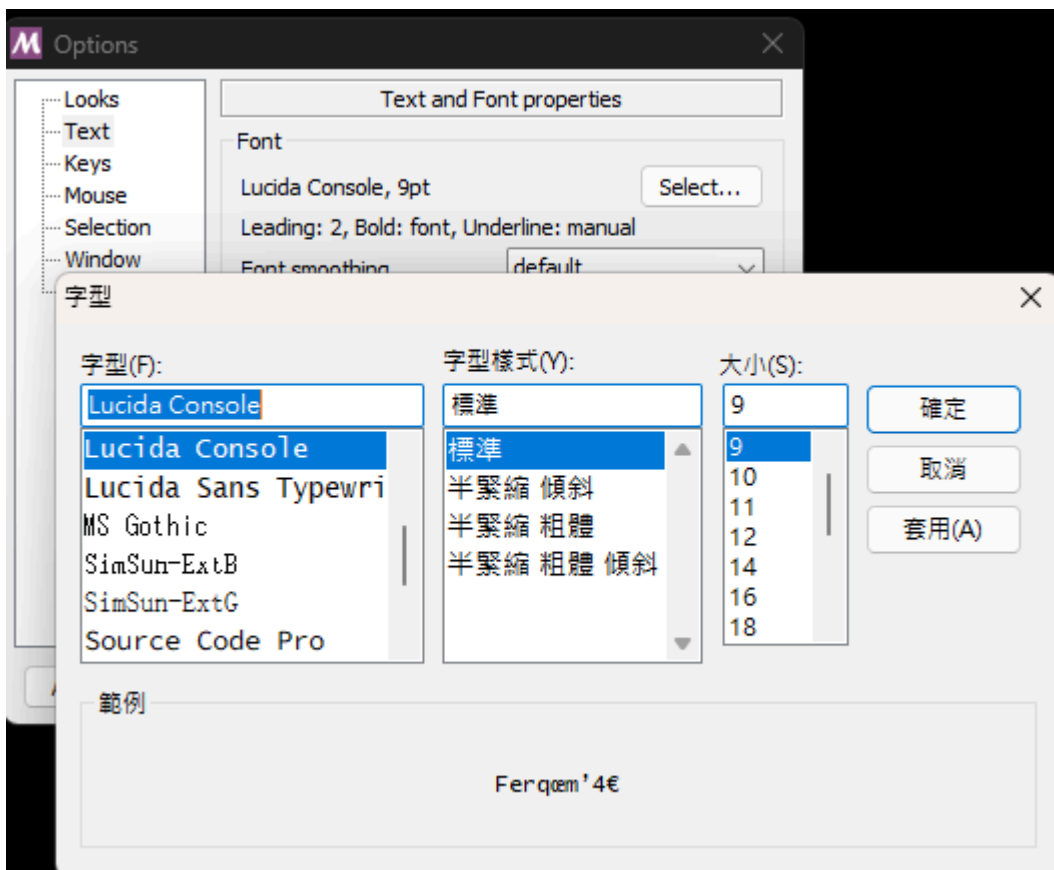


從這張圖可以看到你這段時間累積的成果：

- `ZHENCHENGVISION/`：剛建立的父母攝影素材站。
- `SiongHing/`：這應該就是你之前提到的「上興棉被行」(Shangxing) 專案
- `Portfolio/`：你的個人作品集。
- `sechskies/`：水晶男孩的粉絲網站。



調整字級



專案根目錄

- 切換目錄 (Change Directory):** 你執行的 `cd ZHENCHENGVISION` 動作叫做「**進入工作空間**」。
- 分支標示 (Branch Indicator):** 後方的 `(main)` 表示你目前正處於 Git 的「**主分支**」(Main Branch)。這意味著這個資料夾已經成功初始化為一個 Git 倉庫 (Repository)，並且 Git 正在監控這個目錄下的所有變動。

- **工作區 (Working Directory):** 你現在所在的位置就是 「工作區」。你接下來新增的 HTML、CSS 或照片素材，都會先存在這裡。

```
User@1113-14 MINGW64 /d/Developer/projects
$ cd ZHENCHENGVISION

User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
$
```

檢視指令紀錄

執行 history 指令的動作，專業用語稱為 「檢視指令紀錄」 (Viewing Command History)。

這在開發中非常實用，特別是當你忘記之前輸入過什麼複雜指令，或者想確認操作順序時。

從你的紀錄中，我觀察到一個很有趣的操作點：

1. 指令紀錄分析：

- **第 1 條：** git clone ... 這是標準的複製動作，它會自動建立一個跟倉庫同名的資料夾 ZHENCHENGVISION。
- **第 2 條：** git clone ... 260422
 - **專業用語：** 這叫做 「指定目錄名稱克隆」 (Cloning into a specific directory)。
 - 你在網址後面加了 260422，這告訴 Git：「把東西下載下來，但資料夾名稱我要改叫 260422」。這解釋了為什麼你上一張圖會出現那個日期資料夾！

2. 關於 history 的進階小技巧：

既然你開始使用 history 了，這裡有兩個「工程師級」的快捷鍵分享給你：

- **快速執行：** 如果你想再次執行紀錄中的第 4 條指令 (cd ZHENCHENGVISION)，不需要重新輸入，只需要輸入 !4 然後按 Enter 即可。
- **反向搜尋：** 按下鍵盤的 Ctrl + R，然後開始打「git」，它會自動幫你找回上次輸入過的 git 相關指令。

小提醒： 你現在已經成功進入 ZHENCHENGVISION (main) 了。如果你之前電腦裡已經有寫好的程式碼（例如在 Portfolio 那邊寫的相關頁面），你可以現在把那些檔案「搬進」這個資料夾，然後準備進行第一次的 git add 。

```
User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
$ history
1  git clone https://github.com/jekkinooopy/ZHENCHENGVISION.git
2  git clone https://github.com/jekkinooopy/ZHENCHENGVISION.git 260422
3  ls
4  cd ZHENCHENGVISION
5  history

User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
$ |
```

Git 工作流 (Git Workflow)

1. 暫存變更 (Staging)

- **動作：** git add .
- **專業術語：** 「將變更加入暫存區」 (Adding changes to the Staging Area)。
- **意義：** 這就像是在拍照前，先把所有要入鏡的演員（檔案）叫到舞台上站好。

2. 提交存檔 (Committing)

- **動作：** git commit -m "你的註解"
- **專業術語：** 「建立提交紀錄」 (Creating a Commit)。
- **意義：** 拍下照片並在相簿標註日期與重點。這會產生一個唯一的 「提交雜湊值」 (Commit Hash)，讓你未來可以隨時回溯到這個版本。

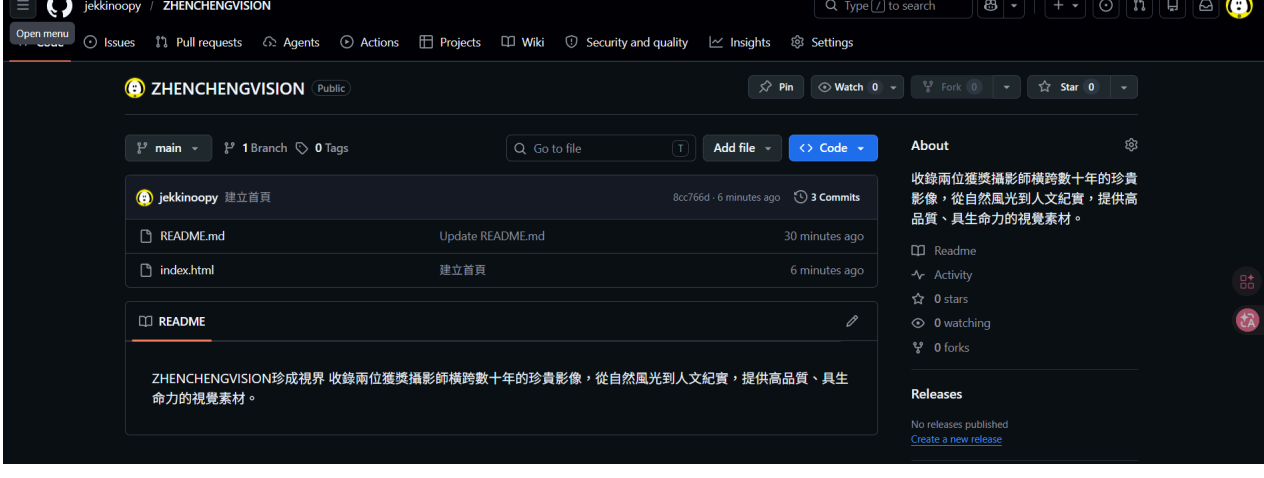
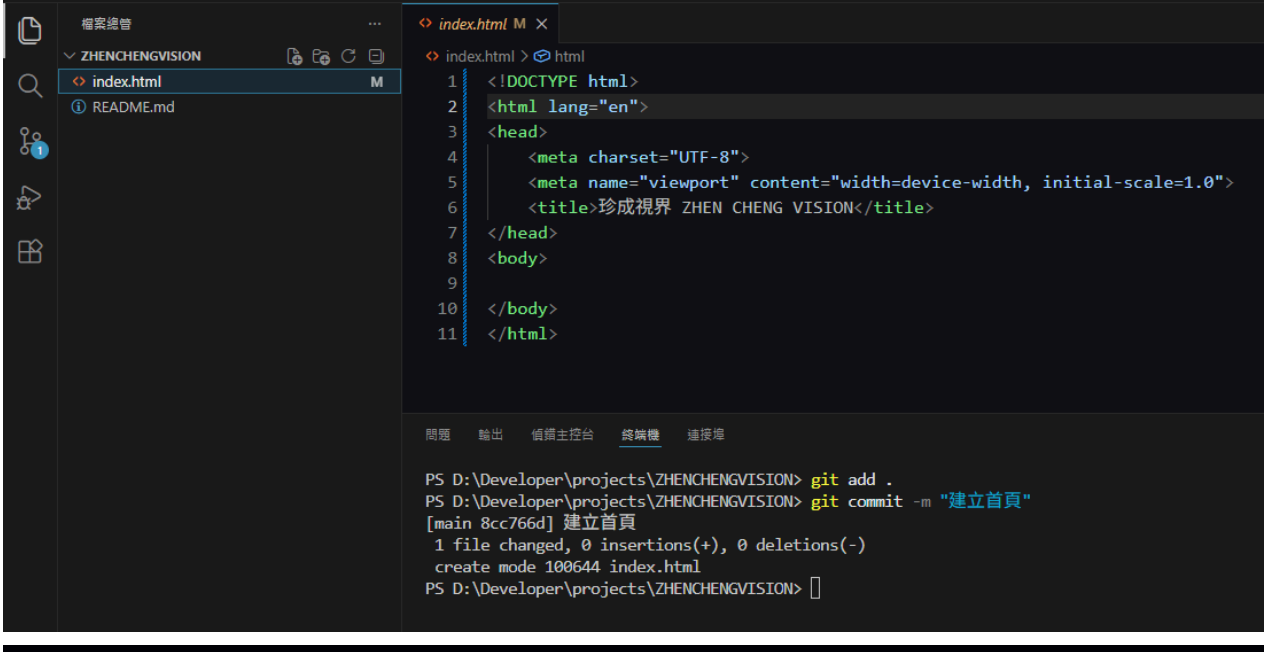
3. 同步遠端 (Pushing)

- **動作：** git push
- **專業術語：** 「推送至遠端倉庫」 (Pushing to Remote Repository)。
- **意義：** 將你電腦裡的「相簿」備份到 GitHub 雲端。

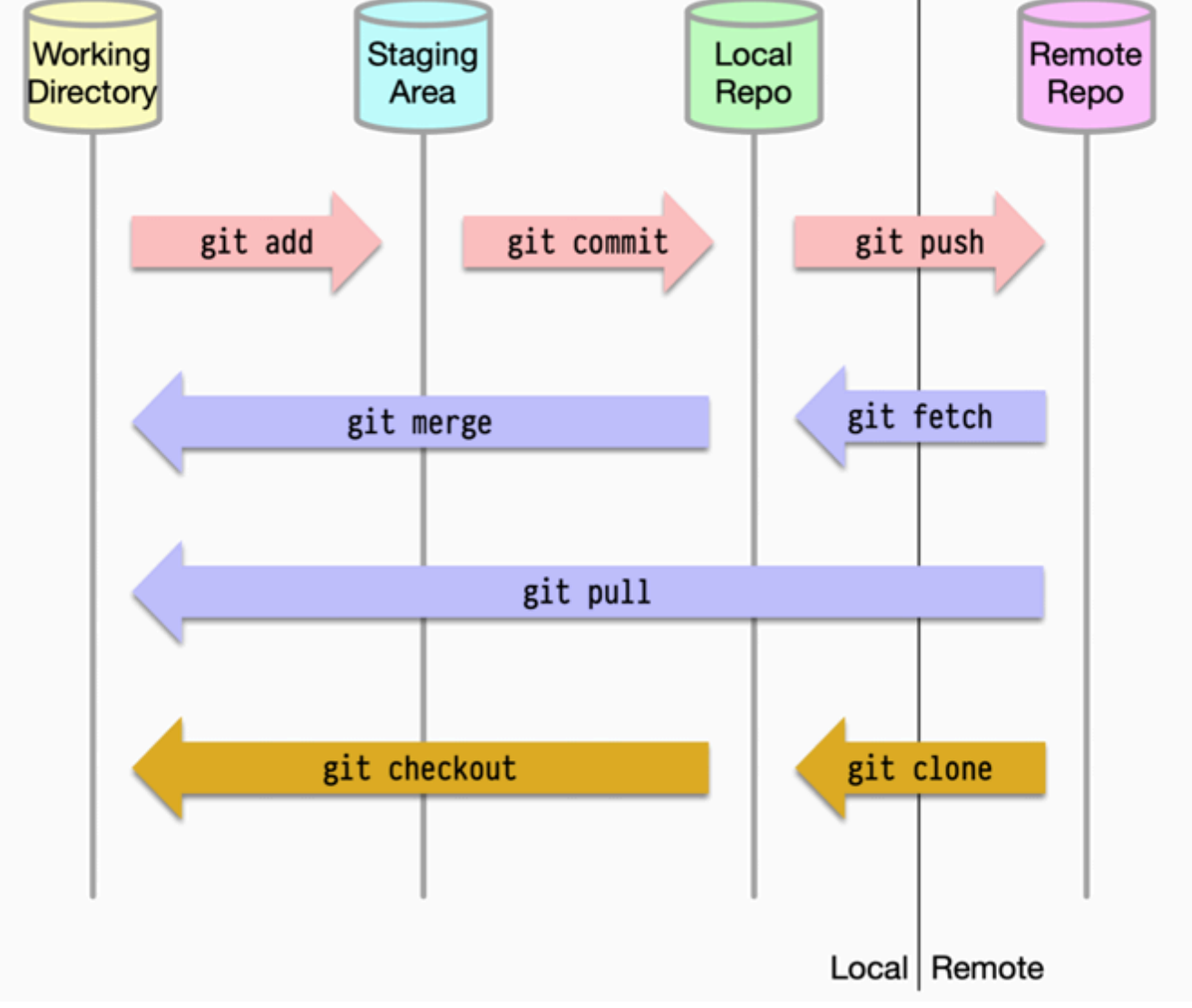
💡 專業整合圖解：Git 的三個區域

理解這三個動作的整合，關鍵在於明白檔案在三個空間的移動：

1. **工作區 (Working Directory)：** 你正在編輯檔案的地方（例如 VS Code 畫面）。
2. **暫存區 (Staging Area)：** 執行 add 後，檔案待命的地方。
3. **本地倉庫 (Local Repo)：** 執行 commit 後，檔案正式紀錄的地方。
4. **遠端倉庫 (Remote Repo)：** 執行 push 後，檔案到達 GitHub 的地方。



Git 工作流程圖 (Git Workflow)



看到了你正在編輯的 style.css 檔案，這是一個很棒的時機來練習 Git。根據你提供的 VS Code 畫面，左側檔案清單旁的 「U」（Untracked）表示這些檔案目前還沒被 Git 追蹤，而 「M」（Modified）表示檔案已被修改。

「Pasted image 20260422114600.png」 無法找到。

要在終端機（Terminal）示範 Git 的狀態切換，你可以按照以下步驟操作：

1. 查看目前的狀態：git status

首先，在 VS Code 下方的終端機輸入：

Bash

```
git status
```

你會看到：

- **Untracked files:** style.css 和 ZHENCHENGVISION.ai 會顯示為紅色，因為 Git 還不認識它們。

- **Changes not staged for commit:** index.html 如果有修改過也會顯示在這裡。

```
User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
$ git status
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  ZHENCHENGVISION.ai
  style.css

no changes added to commit (use "git add" and/or "git commit -a")

User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
$
```

2. 將檔案加入暫存區：git add

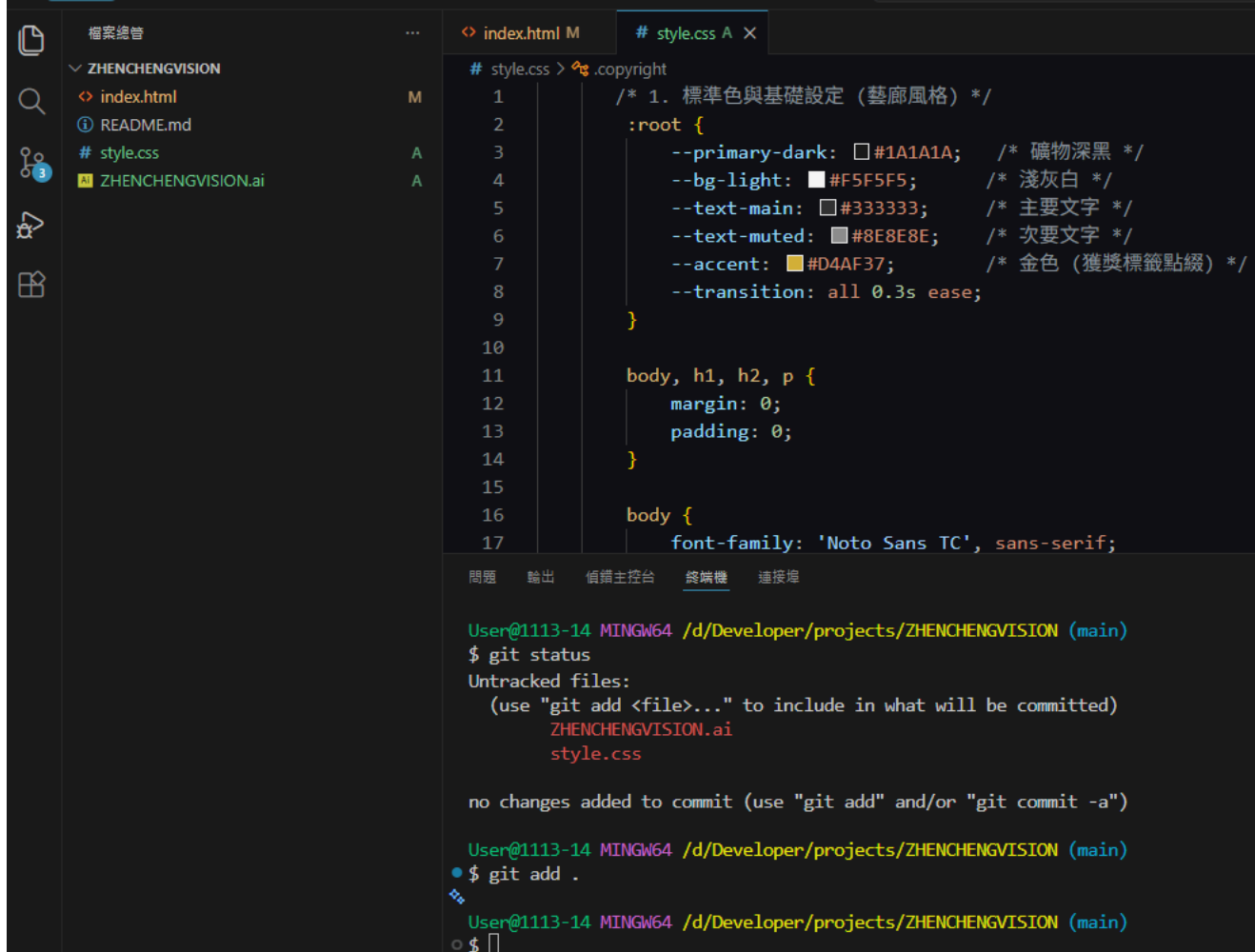
接著，把你想上傳的檔案交給 Git 暫存。

Bash

```
git add style.css index.html
```

(或者使用 `git add .` 一次加入目錄下所有變更)

「Pasted image 20260422115053.png」無法找到。



這時候再輸入 `git status`：你會發現檔案顏色變成了綠色，並顯示在 `Changes to be committed` 欄位下。這代表檔案已經進入了上一張圖提到的 **Staging Area (暫存區)**。

```
User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   ZHENCHENGVISION.ai
    modified:   index.html
    new file:   style.css

User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
$
```

3. 提交變更：git commit

確認暫存區的東西沒問題後，把它們存入本地倉庫（Local Repo）。

Bash

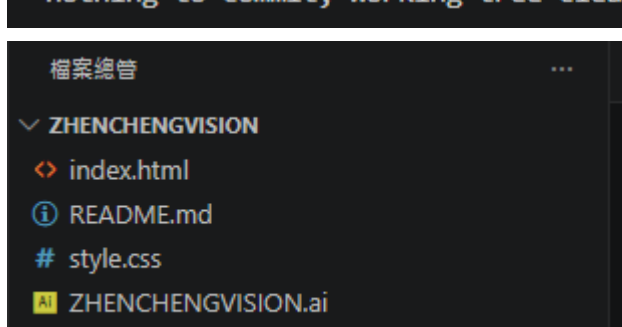
```
git commit -m "建立外部css"
```

```
User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
$ git commit -m "建立外部css"
[main 62ab98a] 建立外部css
3 files changed, 9690 insertions(+)
create mode 100644 ZHENCHENGVISION.ai
create mode 100644 style.css
```

提交後的狀態：執行完後，再打一次 `git status`，它會告訴你 `nothing to commit, working tree clean`。這代表目前工作區的所有變更都已經安全地存入 Git 的歷史紀錄中了。

```
User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```



💡 VS Code 介面小提示

- **U (Untracked):** 檔案是新的，Git 還沒追蹤它。
- **M (Modified):** 檔案被改過了，但還沒 `add` 到暫存區。
- **A (Added):** 你執行了 `git add` 後，在 VS Code 的原始碼控制面板會看到狀態變成 A。

版本節點追蹤與檔案還原

在 VS Code 中操作 Git 時，理解「紀錄」與「時光倒流」是非常重要的。延續你剛才的流程，當你完成 `git commit` 後，這些紀錄就會變成一個個節點。

以下是在 VS Code 終端機與介面中，針對這幾個指令的操作示範與解析：

1. 查看歷史紀錄：git log

當你想看過去做了哪些 commit，輸入：

Bash

```
git log
```

```
User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
$ git log
commit 7dba77bbcab784bfbcc9f4b6e508b02ead76b5c (HEAD -> main)
Author: jekkinoopy <jekkinoopy@gmail.com>
Date:   Wed Apr 22 13:09:37 2026 +0800

    加入css連結

commit 62ab98a76528d450d5de9426f48d4c7c9f238447
Author: jekkinoopy <jekkinoopy@gmail.com>
Date:   Wed Apr 22 11:54:15 2026 +0800

    建立外部css

commit 8cc766d1d192bfbfa6f14fade7ad8c5a6707883a2 (origin/main, origin/HEAD)
Author: jekkinoopy <jekkinoopy@gmail.com>
Date:   Wed Apr 22 10:55:23 2026 +0800

    建立首頁

commit ef946dc7d5b4be2104d7a26405240bd38353e8a2
Author: JekkiNoopy <jekkinoopy@gmail.com>
Date:   Wed Apr 22 10:31:22 2026 +0800

    Update README.md

commit 35cb64e917eeea2011632a8d54dc1806885f00f7
Author: JekkiNoopy <jekkinoopy@gmail.com>
Date:   Wed Apr 22 10:29:04 2026 +0800

    Initial commit
```

```
git log --oneline
```

```
User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
• $ git log --oneline
7dba77b (HEAD -> main) 加入css連結
62ab98a 建立外部css
8cc766d (origin/main, origin/HEAD) 建立首頁
ef946dc Update README.md
35cb64e Initial commit
```

- `--oneline` 會讓紀錄縮短成一行，方便閱讀。
- 你會看到一串 7 位數的英文數字組合（例如 `a1b2c3d`），那就是**提交哈希值 (Commit ID)**，也就是該節點的身份證。
-

2. 移動到特定節點：`git checkout <commit_id>`

如果你想「穿越」回去看某個舊版本（例如還沒改 CSS 之前的樣子）：

Bash

```
git checkout a1b2c3d
```

```
User@1113-14 MINGW64 /d/Developer/projects/ZHENCHENGVISION (main)
• $ git checkout 62ab98a
Note: switching to '62ab98a'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 62ab98a 建立外部css
```

🌪 Git 實戰：版本切換與還原

1. 為什麼要這樣做？

- **查看舊程式碼**：回去看看當時「建立外部 CSS」時的程式碼是怎麼寫的。
- **實驗性修改**：可以在這個舊版本上亂改、測試，而不會弄壞你目前最新的網頁。

2. 圖中關鍵訊息解析

- `HEAD is now at 62ab98a...`：代表切換成功，你的 VS Code 現在顯示的是那個時間點的內容。
- `git switch -c <new-branch-name>`：Git 提示你，如果你在舊版本改一改覺得很棒，可以用這個指令開一個「分身」分支。
- `git switch -`：快速切換回上一個所在的狀態（通常是 `main` 分支）。

🔗 **實戰技巧：如何「回現代」？** 如果你已經看夠了舊版本，想回到最新的進度，請輸入：

Bash

```
git checkout main
```

輸入完後，終端機後方的 `((62ab98a...))` 就會變回 `(main)`，代表你已經平安回到現代了。

3. 檔案回復（還原）

根據檔案所在的階段，還原方式不同：

A. 還原「修改中」但還沒 `add` 的檔案

如果你剛才寫亂了 `style.css`，想直接放棄這次修改：

Bash

```
git checkout -- style.css
```

- **VS Code 快捷法**：在左側「原始碼控制」面板，對著檔案點擊 **右鍵 -> 放棄變更 (Discard Changes)**。

B. 從暫存區 (Staging) 退回

如果你已經 `git add` 了，但反悔不想 `commit` 它：

Bash

```
git reset HEAD style.css
```

- **VS Code 快捷法**：點擊檔案旁邊的「減號 -」(**Unstage Changes**)。

4. 總結：節點的概念

你可以把 Git 想像成一個火車路線圖：

- **Commit**：是一個車站（節點）。
- **Branch (分支)**：是鐵路的分叉點。
- **Checkout**：就是你的火車頭移動到哪一個車站。

💡 修改重點解析（為什麼這樣改比較正）：

1. **代碼塊閉合**：原本你的 `Bash` 字樣在 `<code>` 之外，導致渲染失敗。現在我將它改成 `bash`，這樣 `git checkout main` 就會被包在漂亮的深色方框裡，且有語法高亮。
2. **層次結構**：使用了 `###` 和 `####` 分級，這樣在 Obsidian 的左側「大綱 (Outline)」面板裡，你可以快速跳轉到不同章節。
3. **Callout 區塊**：使用 `> [!tip]` 把「回現代」包起來，這在視覺上會產生一個有顏色的提示框，讓你一眼看到重點。
4. **Inline Code**：用 `<code>` 包住指令（例如 `git add`），會讓指令在一段文字中呈現特殊的底色。